Report

1.1

Customer

Ring Protocol

Aggregator Hook

June 14, 2026

ABDK Consulting

Contents

1. Document Revisions

#	Date	Author	Description
0.1	June 5, 2026	ABDK	Initial draft report issued to the client.
1.0	June 13, 2026	ABDK	Public report issued after review of the client's fixes
1.1	June 14, 2026	ABDK	System overview and conclusion revised.

2. Introduction

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

The *Ring Aggregator Hook* project submitted for review consists of three Solidity files totalling approximately 400 nSLOC: the main Uniswap v4 hook (`RingAggregatorHook.sol`), the fee-forwarding adapter (`RingUniBurner.sol`), and a pure pricing library (`lib/FewV2Math.sol`).

RingAggregatorHook

`RingAggregatorHook` is a Uniswap v4 hook that exposes existing Ring FewV2 liquidity as Uniswap v4 pools without migrating or moving any on-chain liquidity. It inherits from `BaseHook`, `DeltaResolver`, and OpenZeppelin's `ReentrancyGuard`.

The contract is deliberately ownerless: there is no owner, no pause mechanism, and no upgrade path. Every routing parameter (`fewFactory`, `fewV2Factory`, `weth`, `feeRecipient`, `uniBurner`) is declared `immutable` and validated against the zero address in the constructor. The only mutable state is a one-time maximum-approval cache (`_approved`) used to avoid repeated `approve` calls.

The hook enables four permissions: `beforeInitialize`, `beforeSwap`, `beforeSwapReturnDelta`, and `beforeAddLiquidity` (the last always reverts with `LiquidityNotAllowed`, enforcing that liquidity is sourced exclusively from FewV2 pairs).

Pool initialization. `_beforeInitialize` validates that the pool's token pair has corresponding FewTokens registered in `fewFactory` and that a direct FewV2 pair exists in `fewV2Factory`. It additionally rejects 1:1 wrap pairs (those belong to a separate `FewTokenHook`).

Swap execution. `_beforeSwap` (protected by `nonReentrant`) dispatches to one of two internal paths based on the sign of `amountSpecified`:

- **Exact-input** (`amountSpecified < 0`): the absolute value is derived, the input is taken from the PoolManager, wrapped 1:1 into the corresponding FewToken via `IFewWrappedToken.wrap`, routed through the direct FewV2 pair via `_executeFewV2Hop`, and the output FewToken is split: 5 bps are forwarded to `uniBurner` via `_skimUniBurnFee`, and the remainder is unwrapped back to the requested underlying before being settled to the PoolManager.

- **Exact-output** (`amountSpecified > 0`): the required FewToken output is grossed up so that after the 5 bps skim the user still receives the requested `amountOut`. The gross-up uses the formula `(amountOut * FEE_DENOM + (FEE_DENOM - PROTOCOL_FEE_BPS) - 1) / (FEE_DENOM - PROTOCOL_FEE_BPS)`. The required input is back-calculated with `FewV2Math.getAmountIn`. A post-swap check ensures the unwrapped output meets the requested amount.

After every swap a `SwapAggregated` event is emitted recording the pool, router, originating address, direction, specified amount, actual in and out, and gross FewToken output.

Fee collection. A constant 5 bps protocol fee (`PROTOCOL_FEE_BPS = 5`, denominator `FEE_DENOM = 10 000`) is applied to the gross FewToken output on every swap by transferring the fee amount to the immutable `uniBurner` address.

Sweep. `sweep(token)` is permissionless and forwards the contract's entire balance of `token` (ERC-20 or native ETH) to the immutable `feeRecipient`. The destination is fixed at deployment time and cannot be changed.

FewV2 routing. `_executeFewV2Hop` transfers the input FewToken directly to the pair contract and calls `ISwapV2Pair.swap`, following the low-level Uniswap V2 callback pattern. Reserve validation enforces a `MIN_PAIR_RESERVE` sentinel of 1000 wei to detect fully-drained pairs. `_quoteAmountInForHop` performs a read-only quote using `FewV2Math.getAmountIn` without executing a trade, used exclusively for exact-output gross-up.

RingUniBurner

`RingUniBurner` is a push-source fee adapter that accumulates the 5 bps FewToken fees forwarded by `RingAggregatorHook` and converts them into underlying ERC-20 tokens destined for Uniswap's canonical `TokenJar` fee collector. It inherits `Ownable2Step` and `ReentrancyGuard`.

Immutable fields: `tokenJar` (the chain-appropriate Uniswap TokenJar address) and `fewFactory` (for FewToken validation). The single mutable owner-controlled field is `flushPaused`, a boolean pause flag.

- `flush(fewToken)` — permissionless. Validates `fewToken` against `fewFactory`, unwraps the contract's entire accumulated FewToken balance to the underlying ERC-20 (asserting the 1:1 Few protocol invariant), and forwards the underlying to `tokenJar`. Protected by `nonReentrant`.
- `setFlushPaused(bool)` — owner-only. Halts or resumes flushing.

- `emergencyWithdraw(address token, address to)` — owner-only. Allows the owner to rescue any ERC-20 token (or native ETH via `token == address(0)`) stuck in the contract and route it via an alternative path. Protected by `nonReentrant`.
- `renounceOwnership()` — permanently blocked (reverts), preserving access to the pause and emergency-rescue functions.

FewV2Math

`FewV2Math` is a stateless Solidity library providing two pure pricing functions for Uniswap V2-style pairs with a 0.3% swap fee:

- `getAmountOut(amountIn, reserveIn, reserveOut)` — classic V2 output formula: $\text{out} = \frac{\text{in} \times 997 \times R_{\text{out}}}{R_{\text{in}} \times 1000 + \text{in} \times 997}$.
- `getAmountIn(amountOut, reserveIn, reserveOut)` — V2 input formula with canonical $+1$ ceiling rounding: $\text{in} = \left\lfloor \frac{R_{\text{in}} \times \text{out} \times 1000}{(R_{\text{out}} - \text{out}) \times 997} \right\rfloor + 1$, added for exact-output support.

Both functions revert with `InsufficientAmount` on zero input/output and with `InsufficientLiquidity` on zero or insufficient reserves.

/ (root directory)

AUDIT_SCOPE.md KNOWN_ISSUES.md README.md SECURITY.md

/src/

RingAggregatorHook.sol RingUniBurner.sol

/src/lib/

FewV2Math.sol

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the Aggregator Hook project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment
The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis
Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis
For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis
The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical
Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major
Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate
Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor
Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the Aggregator Hook codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Major	1	1
Moderate	12	8

Minor	Found	Fixed
	34	8

Fixed 17 out of 47 issues reported.

5.1. Major

CVF-1: fixed Category: **over-underflow**

Underflow is possible when converting to `uint256`.

Recommendation

Use safe conversion.

/src/RingAggregatorHook.sol

```
278         uint256 amountIn = uint256(-params.amountSpecified);
```

5.2. Moderate

CVF-2: acknowledged Category: **documentation**

These errors could be made more useful by adding certain paramet

/src/RingAggregatorHook.sol

```
71     error InvalidPoolFee();
72     error UseFewTokenHookForWrapPairs();
73     error NoFewV2Route();
```

```
74      error LiquidityNotAllowed();
77      error EthForwardFailed();
```

/src/lib/FewV2Math.sol

```
11      error InsufficientAmount();
12      error InsufficientLiquidity();
```

CVF-3: fixed Category: **behavior**

It is unclear why zero amount is considered as an error.

/src/lib/FewV2Math.sol

```
20          if (amountIn == 0) revert InsufficientAmount();
```

same file

```
35          if (amountOut == 0) revert InsufficientAmount();
```

CVF-4: fixed Category: **behavior**

Adding 1 is not the same as ceil rounding.

/src/lib/FewV2Math.sol

```
28      /// @notice Input required for a given output. Classic V2:
amountIn = (Rin * out * 1000) / ((Rout - out) * 997) + 1
29      /// @dev `+1` is the V2-canonical ceil-rounding to prevent
under-sourcing the pair.
40          amountIn = (numerator / denominator) + 1;
```

CVF-5: acknowledged Category: **procedural**

Different access level is used for the numerator and denominator

/src/RingAggregatorHook.sol

```
84      /// @notice 5 bps of every swap's gross fewToken output is
forwarded to `uniBurner`.
```

```
85      uint24 public constant PROTOCOL_FEE_BPS = 5;
86      uint256 private constant FEE_DENOM = 10_000;
```

CVF-6: acknowledged Category: **readability**

This function processes each argument separately, so it doesn't

/src/RingAggregatorHook.sol

```
218     function _defaultFewPair(address t0, address t1) internal view
returns (address fewA, address fewB) {
219         address u0 = t0 == address(0) ? address(weth) : t0;
220         address u1 = t1 == address(0) ? address(weth) : t1;
221         fewA = fewFactory.getWrappedToken(u0);
222         fewB = fewFactory.getWrappedToken(u1);
223     }
```

CVF-7: acknowledged Category: **readability**

This code is basically the same in both branches.

/src/RingAggregatorHook.sol

```
350             _ensureApproval(address(weth), fewToken);
351             uint256 minted = IFewWrappedToken(fewToken).wrap(amount);
352             if (minted != amount) revert WrapMismatch(amount, minted);
353             return minted;
356         _ensureApproval(underlying, fewToken);
357         uint256 m = IFewWrappedToken(fewToken).wrap(amount);
358         if (m != amount) revert WrapMismatch(amount, m);
359         return m;
```

same file

```
364             uint256 underlyingAmt =
IFewWrappedToken(fewToken).unwrap(fwAmount);
365             if (underlyingAmt != fwAmount) revert
UnwrapMismatch(fwAmount, underlyingAmt);
367             return underlyingAmt;
369         uint256 amt = IFewWrappedToken(fewToken).unwrap(fwAmount);
370         if (amt != fwAmount) revert UnwrapMismatch(fwAmount, amt);
```

```
371         return amt;
```

CVF-8: fixed Category: **documentation**

Missing documentation

/src/RingAggregatorHook.sol

```
433                 (bool ok,) = payable(feeRecipient).call{value:
amount}("");
434                 if (!ok) revert EthForwardFailed();
```

CVF-9: fixed Category: **documentation**

Here a particular constant value is mentioned in a comment, which is a bad practice, as such comment can easily get out of sync with the code.

Recommendation

Declare a named constant for the value and mention the constant name instead.

/src/RingUniBurner.sol

```
14 /// @notice Receives the 5 bps protocol fee (denominated in
fewTokens) from
```

CVF-10: fixed Category: **documentation**

Mentioning particular deployment details in a comment looks odd, as the code in general doesn't know where and how it will be deployed.

Recommendation

Mention a link to a web page where deployment details are supposed to be published. If some values, such as mainnet addresses, ought to always match the actual deployment, implement public getters in the contract for this values, so user could read these values using blockchain explorers.

/src/RingUniBurner.sol

```
31 ///           Ethereum mainnet:
   0xf38521f130fcCF29dB1961597bc5d2B60F995f85
32 ///           Also live on:     Arbitrum One, Base, OP Mainnet,
   Polygon,
33 ///                             Unichain, World Chain, Celo, Zora,
   Soneium,
34 ///                             X Layer.
```

CVF-11: fixed Category: **documentation**

The former comment sounds like flush can always be triggered by anyone, while actually the owner can pause flushing.

Recommendation

Rephrase the comment to explicitly mention, that flushes could be paused by the owner.

/src/RingUniBurner.sol

```
40 ///         2. PERMISSIONLESS FLUSH — `flush(fewToken)` is callable
   by anyone.
41 ///            No slippage, no oracle dependency: we just unwrap (1:1)
   and
42 ///            push the underlying to TokenJar. Keeper bots can run it
   on
43 ///             a cron without worrying about MEV.
```

```
67     /// @notice Pause flag. Owner can disable flush entirely if
   TokenJar is
68     ///         temporarily paused (e.g., during a Foundation
   migration).
69     bool public flushPaused;
```

CVF-12: fixed Category: **behavior**

This function doesn't allow rescuing pure ether. Even if a contract doesn't implement any payable functions, it is still possible to send ether to it via `selfdestruct` operation.

Recommendation

Implement ability to rescue ether as well.

/src/RingUniBurner.sol

```
137     /// @notice Emergency withdraw of any ERC20 stuck in this
contract.
144     function emergencyWithdraw(address token, address to) external
onlyOwner {
145         if (to == address(0)) revert ZeroAddress();
```

CVF-13: fixed Category: **flaw**

emergencyWithdraw lacks reentrancy guard

`RingUniBurner.emergencyWithdraw` makes an external ERC-20 call (`IERC20(token).safeTransf amount)`) without a `nonReentrant` modifier, violating the checks-effects-interactions pattern. All other state-changing functions in the contract (`flush`) are protected by `non-Reentrant`, but `emergencyWithdraw` is not.

If `token` is an ERC-777-style token (or any token with a receiver hook on `to`), the transfer callback can re-enter `emergencyWithdraw` before the first transfer completes. On re-entry, `balanceOf(address(this))` still returns the full pre-transfer balance, so the re-entrant call attempts to transfer the same amount a second time. The second transfer would fail due to insufficient balance, causing the inner call to revert and the outer call to proceed — so the vulnerability does not result in a double-spend under standard ERC-20 semantics. However, it remains a violation of CEI, creates an exploitable code path under non-standard token behaviour, and contradicts the Slither triage in `docs/SLITHER_TRIAGE.md` which (incorrectly) treated the `reentrancy-events` hit on `emergencyWithdraw` as a false positive by citing `nonReentrant` protection that does not in fact exist on this function.

Recommendation

Add the `nonReentrant` modifier to `emergencyWithdraw`, consistent with `flush`.

```
function emergencyWithdraw(address token, address to) external
  onlyOwner nonReentrant {
```

/src/RingUniBurner.sol

```
144     function emergencyWithdraw(address token, address to) external
onlyOwner {
```

5.3. Minor

CVF-14: acknowledged Category: **readability**

Brackets are redundant here.

/src/RingAggregatorHook.sol

```
337         uint256 fee = (fwOutAmount * PROTOCOL_FEE_BPS) / FEE_DENOM;
```

/src/lib/FewV2Math.sol

```
24          uint256 denominator = (reserveIn * 1000) + amountInWithFee;
```

CVF-15: fixed Category: **readability**

The values `997` and `1000` should be named constants.

/src/lib/FewV2Math.sol

```
22          uint256 amountInWithFee = amountIn * 997;
24          uint256 denominator = (reserveIn * 1000) + amountInWithFee;
```

same file

```
38          uint256 numerator = reserveIn * amountOut * 1000;
39          uint256 denominator = (reserveOut - amountOut) * 997;
```

CVF-16: fixed Category: **documentation**

Mentioning particular constant values in comments is a bad practice as such comments could easily get out of sync with the code.

Recommendation

Declare named constants for the values and use constant names in comments.

/src/lib/FewV2Math.sol

```
14    /// @notice Output for a given input. Classic V2: amountOut =
amountIn * 997 * Rout / (Rin*1000 + amountIn*997)
28    /// @notice Input required for a given output. Classic V2:
amountIn = (Rin * out * 1000) / ((Rout - out) * 997) + 1
```

CVF-17: fixed Category: **procedural**

This version requirement is inconsistent with other files.

Recommendation

Use consistent version requirements across code base.

/src/RingAggregatorHook.sol

```
2 pragma solidity 0.8.26;
```

CVF-18: fixed Category: **procedural**

Specifying a particular compiler version makes it harder migrating to newer versions.

Recommendation

Specify as `^0.8.0` or as `^0.8.26` if there is something special regarding this particular version.

/src/RingAggregatorHook.sol

```
2 pragma solidity 0.8.26;
```

/src/RingUniBurner.sol

```
2 pragma solidity 0.8.26;
```

CVF-19: acknowledged Category: **procedural**

We didn't review these files.

/src/RingAggregatorHook.sol

```
21 import {IFewWrappedToken} from
"./interfaces/external/IFewWrappedToken.sol";
22 import {IFewFactory} from "./interfaces/external/IFewFactory.sol";
23 import {ISwapV2Pair, ISwapV2Factory} from
"./interfaces/external/IFewV2.sol";
```

/src/RingUniBurner.sol

```
10 import {IFewFactory} from "./interfaces/external/IFewFactory.sol";
11 import {IFewWrappedToken} from
"./interfaces/external/IFewWrappedToken.sol";
```

CVF-20: acknowledged Category: **naming**

The name of this error doesn't sound like an error.

Recommendation

Rename, for example to `WrapPairNotAllowed`.

/src/RingAggregatorHook.sol

```
72     error UseFewTokenHookForWrapPairs();
```

CVF-21: acknowledged Category: **typing**

The type for the `pair` parameter should be `ISwapV2Pair`.

/src/RingAggregatorHook.sol

```
75     error TokenMismatch(address pair);
81     error DegeneratePair(address pair);
```

CVF-22: acknowledged Category: **efficiency**

In Solidity, narrower types, such as `uint24` are not more efficient than full-word types, such as `uint256`.

Recommendation

Use `uint256` type.

/src/RingAggregatorHook.sol

```
85    uint24 public constant PROTOCOL_FEE_BPS = 5;
```

CVF-23: acknowledged Category: **typing**

The type for the first key in this mapping should be `IERC20`.

/src/RingAggregatorHook.sol

```
114   /// @notice Tracks (token, fewToken) approvals — one-time max approval cache. Internal only.
115   mapping(address => mapping(address => bool)) internal _approved;
```

CVF-24: acknowledged Category: **typing**

The type for this field should be `ISwapV2Pair`.

/src/RingAggregatorHook.sol

```
120       address pair;
```

CVF-25: acknowledged Category: **typing**

The type for these fields should be more specific.

/src/RingAggregatorHook.sol

```
118       address fewIn;
119       address fewOut;
```

CVF-26: acknowledged Category: **typing**

The type for this parameter should be more specific.

/src/RingAggregatorHook.sol

```
126         address indexed router,
```

CVF-27: acknowledged — Category: typing

The type for the `token` parameter should be `IERC20`.

/src/RingAggregatorHook.sol

```
134     event Sweep(address indexed token, address indexed to, uint256
amount, address indexed triggeredBy);
```

/src/RingUniBurner.sol

```
82     event EmergencyWithdrawn(address indexed token, address indexed
to, uint256 amount);
```

CVF-28: acknowledged — Category: typing

The type for the `newToken` parameter should be `IERC20`.

/src/RingAggregatorHook.sol

```
135     event UniFeeAccrued(PoolId indexed poolId, address indexed
fewToken, uint256 amount);
```

CVF-29: open — Category: readability

These checks are redundant, as a dead address can be passed anyway.

Recommendation

Remove these checks.

/src/RingAggregatorHook.sol

```
146         if (address(_fewFactory) == address(0)) revert ZeroAddress();
```

```
147         if (address(_fewV2Factory) == address(0)) revert
ZeroAddress();
148         if (address(_weth) == address(0)) revert ZeroAddress();
149         if (_feeRecipient == address(0)) revert ZeroAddress();
150         if (_uniBurner == address(0)) revert ZeroAddress();
```

/src/RingUniBurner.sol

```
87          if (_tokenJar == address(0)) revert ZeroAddress();
88          if (_fewFactory == address(0)) revert ZeroAddress();
```

CVF-30: fixed Category: **documentation**

It looks like the `/* payer */` comment belongs to the `uint256 amount` argument, while actually it belongs to the `address` unnamed argument.

Recommendation

Place the comment appropriately.

/src/RingAggregatorHook.sol

```
182         address,
183         /* payer */
184         uint256 amount
```

CVF-31: acknowledged Category: **readability**

The condition `a != address(0)` is checked twice.

Recommendation

Check once at the beginning of the function:

```
if (a == address(0)) return false;
```

/src/RingAggregatorHook.sol

```
213             return a == fewWETH && fewWETH != address(0);
215         return fewFactory.getWrappedToken(b) == a && a != address(0);
```

CVF-32: acknowledged Category: **readability**

Should be `else return` for readability.

/src/RingAggregatorHook.sol

```
215         return fewFactory.getWrappedToken(b) == a && a != address(0);
```

CVF-33: acknowledged Category: **readability**

Should be `else return` for readability.

/src/RingAggregatorHook.sol

```
215         return fewFactory.getWrappedToken(b) == a && a != address(0);
```

CVF-34: acknowledged Category: **readability**

The `params.zeroForOne` condition is checked twice.

Recommendation

Check once:

```
(Currency inCurr, Currency outCurr) =
  params.zeroForOne ?
    (key.currency0, key.currency1) :
    (key.currency1, key.currency0);
```

/src/RingAggregatorHook.sol

```
242         Currency inCurr = params.zeroForOne ? key.currency0 :
key.currency1;
243         Currency outCurr = params.zeroForOne ? key.currency1 :
key.currency0;
```

CVF-35: fixed Category: **efficiency**

Here one is subtracted from a non-constant expression, which prevents compiler from precomputing this operation.

Recommendation

Rearrange the expression to subtract from a constant expression:

```
amountOut * FEE_DENOM + (FEE_DENOM - PROTOCOL_FEE_BPS - 1)
```

/src/RingAggregatorHook.sol

```
310         uint256 fwOutGross =
311             (amountOut * FEE_DENOM + (FEE_DENOM - PROTOCOL_FEE_BPS) -
1) / (FEE_DENOM - PROTOCOL_FEE_BPS);
```

CVF-36: fixed Category: **efficiency**

Here one is subtracted from a non-constant expression, which prevents compiler from precomputing this operation.

Recommendation

Rearrange the expression to subtract from a constant expression:

```
amountOut * FEE_DENOM + (FEE_DENOM - PROTOCOL_FEE_BPS - 1)
```

/src/RingAggregatorHook.sol

```
310         uint256 fwOutGross =
311             (amountOut * FEE_DENOM + (FEE_DENOM - PROTOCOL_FEE_BPS) -
1) / (FEE_DENOM - PROTOCOL_FEE_BPS);
```

CVF-37: acknowledged Category: **typing**

The type for the `fewToken` argument should be `IFewWrappedToken`.

/src/RingAggregatorHook.sol

```
347     function _wrap(Currency inCurr, address fewToken, uint256 amount)
internal returns (uint256) {
```

same file

```
362     function _unwrap(address fewToken, Currency outCurr, uint256
fwAmount) internal returns (uint256) {
```

/src/RingUniBurner.sol

```
100     function flush(address fewToken) external nonReentrant returns
(uint256 amount) {
```

CVF-38: acknowledged Category: **readability**

The rest of the function looks like it is always executed, while actually it is executed only when `inCurr` is not zero address.

Recommendation

Put the rest of the function into an explicit `else` block for readability.

/src/RingAggregatorHook.sol

```
354         }
355         address underlying = Currency.unwrap(inCurr);
356         _ensureApproval(underlying, fewToken);
357         uint256 m = IFewWrappedToken(fewToken).wrap(amount);
358         if (m != amount) revert WrapMismatch(amount, m);
359         return m;
```

same file

```
368         }
369         uint256 amt = IFewWrappedToken(fewToken).unwrap(fwAmount);
370         if (amt != fwAmount) revert UnwrapMismatch(fwAmount, amt);
371         return amt;
```

CVF-39: acknowledged Category: **typing**

The type for the `token` argument should be `IERC20`.

/src/RingAggregatorHook.sol

```
374     function _ensureApproval(address token, address spender) internal
{
```

same file

```
428     function sweep(address token) external nonReentrant {
```

/src/RingUniBurner.sol

```
144     function emergencyWithdraw(address token, address to) external
onlyOwner {
```

CVF-40: acknowledged Category: **typing**

The type for the `pair` argument should be `ISwapV2Pair`.

/src/RingAggregatorHook.sol

```
381     function _executeFewV2Hop(address pair, address tokenIn, address
tokenOut, uint256 amountIn)
```

same file

```
394     function _quoteAmountInForHop(address pair, address tokenIn,
address tokenOut, uint256 finalOut)
```

same file

```
403     function _hopState(address pair, address tokenIn, address
tokenOut)
```

CVF-41: acknowledged Category: **typing**

The type for the token arguments should be `IERC20`.

/src/RingAggregatorHook.sol

```
403     function _hopState(address pair, address tokenIn, address
tokenOut)
```

CVF-42: acknowledged Category: **typing**

The type of the `fewA` and `fewB` return values should be `IFewWrappedToken`.

/src/RingAggregatorHook.sol

```
445     function defaultRouteFor(PoolKey calldata key) external view
returns (address fewA, address fewB, address pair) {
```

CVF-43: acknowledged Category: **typing**

The type for the `pair` return value should be `ISwapV2Pair`.

/src/RingAggregatorHook.sol

```
445     function defaultRouteFor(PoolKey calldata key) external view
returns (address fewA, address fewB, address pair) {
```

CVF-44: fixed Category: **documentation**

It is a good practice to put a comment into an empty block to explain why the block is empty.

/src/RingAggregatorHook.sol

```
453     receive() external payable {}
```

CVF-45: acknowledged Category: **typing**

The type for the `fewToken` parameter should be `IFewWrappedToken`.

/src/RingUniBurner.sol

```
74     error UnknownFewToken(address fewToken);
```

same file

```
80     event Flushed(address indexed fewToken, address indexed
underlying, uint256 amount, address indexed caller);
```

CVF-46: acknowledged Category: **typing**

The type for the `underlying` parameter should be `IERC20`.

/src/RingUniBurner.sol

```
80     event Flushed(address indexed fewToken, address indexed
underlying, uint256 amount, address indexed caller);
```

CVF-47: acknowledged Category: **typing**

The type for the `_fewFactory` argument should be `IFewFactory`.

/src/RingUniBurner.sol

```
86     constructor(address _tokenJar, address _fewFactory, address
_owner) Ownable(_owner) {
```

6. Conclusion

The audit identified 47 issues spread across one high-severity, twelve medium-severity, and thirty-four low-severity findings. Following the client's remediation pass, 17 issues were fixed, 29 were acknowledged (accepted as-is), and 1 remains open.

High-Severity Findings

One high-severity issue was reported and fixed.

CVF-1 (*over-underflow, fixed*) — In the original exact-input path, `uint256(-params.amountSpecified)` was applied without a prior sign-check. For `amountSpecified` $\geq$ `0` this produces an arithmetic underflow in Solidity's checked-arithmetic mode, reverting the swap with an obscure error rather than a meaningful one, and for the maximum negative `int256` the negation itself overflows. The client resolved this by introducing a dedicated `_exactInputAmount()` helper that first calls `amountSpecified.toInt128()` (which enforces the `int128` range and therefore implicitly checks the sign), then uses `OZSafeCast.toUint256` on the negated value.

Medium-Severity Findings

Twelve medium-severity issues were reported; eight were fixed and four were acknowledged.

CVF-3 (*behavior, fixed*) — Zero amounts were rejected by an explicit revert guard in `FewV2Math.getAmountOut`; the defensive check was extended with an `InsufficientAmount` revert on zero output as well.

CVF-4 (*behavior, fixed*) — `getAmountIn` used `+ 1` as a ceiling approximation, which is not a proper ceiling division. The client revised the computation to canonical Uniswap V2 ceiling rounding.

CVF-8 (*documentation, fixed*), **CVF-9** (*documentation, fixed*), **CVF-10** (*documentation, fixed*), **CVF-11** (*documentation, fixed*) — Missing, outdated, or deployment-specific comment text in `RingAggregatorHook` was corrected.

CVF-12 (*behavior, fixed*) — `emergencyWithdraw` in `RingUniBurner` did not handle native ETH. A dedicated `token == address(0)` branch now forwards the contract's ETH balance to the specified recipient, reverting with `EthForwardFailed` on failure.

CVF-13 (*flaw, fixed*) — `emergencyWithdraw` lacked a `nonReentrant` guard despite making an external ERC-20 call, creating a potential reentrancy vector with ERC-777-style tokens. The guard was added.

The following four medium issues were acknowledged without code changes: **CVF-2** (custom errors lacking parameters), **CVF-5** (inconsistent access levels on fee constants), **CVF-6** (function that could be split for clarity), **CVF-7** (duplicated logic across two swap-path branches).

Low-Severity Findings

Thirty-four low-severity issues were reported; eight were fixed, twenty-five were acknowledged, and one remains open.

CVF-15 (*readability, fixed*) — The values `997` and `1000` in `FewV2Math` were promoted to named constants.

CVF-16, **CVF-30**, **CVF-44** (*documentation, fixed*) — Inline comments referencing specific constant values, containing misplaced parameter annotations, or appearing in empty blocks were corrected.

CVF-17 (*procedural, fixed*) — An inconsistent `pragma solidity` specifier in `FewV2Math.sol` was unified to `0.8.26`.

CVF-18 (*procedural, fixed*) — An exact compiler version pin was replaced with a minimum-version pragma.

CVF-35, **CVF-36** (*efficiency, fixed*) — In the exact-output gross-up formula, a `- 1` adjustment was subtracted from a non-constant sub-expression, preventing constant-folding. The client introduced a named constant `USER_FEE_BPS` to improve readability.

The twenty-five acknowledged low-severity findings are predominantly type-precision observations: numerous parameters and return values were typed as `address` where a typed interface (`IERC20`, `ISwapV2Pair`, `IFewWrappedToken`, `IFewFactory`) would be more precise (CVF-21–28, CVF-37–43, CVF-45–47). Also acknowledged: duplicate condition checks and redundant early-returns (CVF-31–34); a `uint24` fee parameter where `uint256` incurs no added cost (CVF-22); an error name that does not read as an error (CVF-20); redundant parentheses (CVF-14); and a note that certain external files were out of scope (CVF-19).

CVF-29 (*readability, open*) — Zero-address constructor guards considered redundant given downstream validation remain in the codebase; the client regards them as intentional defensive checks.

Summary

Severity	Total	Fixed	Acknowledged	Open
High	1	1	0	0
Medium	12	8	4	0
Low	34	8	25	1
Total	**47**	**17**	**29**	**1**

The sole high-severity issue and all critical behavioral and security findings in the medium category were addressed by the client. The remaining open finding is of low severity and informational nature. No unresolved security vulnerabilities are known to exist in the audited codebase at the time of this report.

ABDK
Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

EMail

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

@ABDKConsulting

LinkedIn

company/abdk-consulting